

공 기 업 전 산 직 채 용 대 비

전산직 봉투모의고사

전산 실전모의고사 제1회

정답 및 해설

빠 른 정 답

번호	정답	번호	정답	번호	정답
01	③	15	②	29	①
02	①	16	④	30	③
03	④	17	①	31	④
04	④	18	②	32	②
05	⑤	19	①	33	①
06	②	20	⑤	34	⑤
07	③	21	③	35	①
08	⑤	22	①	36	①
09	④	23	②	37	②
10	④	24	②	38	④
11	②	25	①	39	③
12	④	26	⑤	40	⑤
13	②	27	③		
14	①	28	③		

정답 분포 — ① 10개 / ② 9개 / ③ 7개 / ④ 8개 / ⑤ 6개

전 자 계 산 기 구 조

01 ③ 캐시

이 시스템의 평균 메모리 접근시간은?

계층 구조는 위에서부터 차례로 본다. L1은 어느 접근이든 한 번은 거치므로 1ns가 항상 든다.

L1에서 놓치는 10%만 L2를 본다. 그 10% 가운데 20%가 다시 주기억장치로 간다.

$$1 + 0.10 \times (8 + 0.20 \times 60) = 1 + 0.10 \times 20 = 3.00ns$$

어디서 끝나나	비율	드는 시간	기여
L1 적중	90%	1ns	0.90
L2 적중	$10\% \times 80\% = 8\%$	$1 + 8 = 9ns$	0.72
주기억	$10\% \times 20\% = 2\%$	$1 + 8 + 60 = 69ns$	1.38
합			3.00

선택지 해설

- ① 2.00ns는 L1 적중시간 1ns를 더하지 않은 값이다. L1은 놓치든 맞히든 **언제나 먼저 본다.**
- ② 2.74ns는 세 계층을 동시에 본다고 가정한 값이다 ($0.9 \times 1 + 0.08 \times 8 + 0.02 \times 60$). 계층 구조는 동시 접근이 아니다.
- ③ (정답) $1 + 0.10 \times (8 + 0.20 \times 60) = 3.00ns$.
- ④ 7.80ns는 L2에서 놓치는 비율을 100%로 본 값이다($1 + 0.1 \times 68$). L2도 80%를 잡아낸다.
- ⑤ 13.80ns는 L2 미스율 20%를 **전체 접근 기준**으로 본 값이다($1 + 0.1 \times 8 + 0.2 \times 60$). 20%는 L1을 놓친 10% 안에서의 비율이다.

02 ① 수체계

8비트 2의 보수로 -96 - 45 를 계산한 결과와 오버플로 판정으로 옳은 것은?

-96 = 1010 0000, -45 = 1101 0011 이다(각각 96과 45의 2의 보수).

	2진	10진(부호 없이)
-96	1010 0000	160
-45	1101 0011	211
합	1 0111 0011	371
8비트만	0111 0011	115

참값은 -141인데 8비트 2의 보수가 담을 수 있는 범위는 -128 ~ 127 이다. 담기지 못했다.

음수 둘을 더했는데 결과의 부호비트가 0이다. 이것이 오버플로의 표시다. 8비트를 넘어간 자리올림은 버리는 것이 정상이므로 판정 근거가 되지 못한다.

선택지 해설

- ① (정답) $160 + 211 = 371$, 하위 8비트가 0111 0011 이고 부호비트가 1·1 → 0 이므로 오버플로다.
- ② 1000 1101은 141의 2진 표기를 그대로 적은 것이다. 2의 보수에서 이 패턴은 -115를 뜻한다.
- ③ 1010 0000은 -96 자체의 표현이다. 아직 빠지 않았다.
- ④ 0011 0011은 51이다. 부호를 무시하고 $96 - 45$ 를 계산한 값이다.
- ⑤ 비트 패턴은 맞지만 오버플로를 놓쳤다. 음수 둘의 합이 양수로 나왔다.

운 영 체 제

03 ④ 스케줄링

HRN 기법으로 고를 때 다음에 실행되는 프로세스와 그 응답비는?

HRN의 응답비는 (대기시간 + 서비스시간) ÷ 서비스시간이다. 값이 큰 쪽을 먼저 고른다.

프로세스	식	응답비
A	$(10 + 5) \div 5$	3.00
B	$(15 + 10) \div 10$	2.50
C	$(9 + 3) \div 3$	4.00
D	$(20 + 20) \div 20$	2.00

HRN은 SJF의 무기한 연기를 막으려고 만든 기법이다. 서비스시간이 짧을수록, 그리고 오래 기다렸을수록 응답비가 커진다. 가장 오래 기다린 D가 뽑히지 못하는 것은 서비스시간도 20으로 함께 크기 때문이다.

선택지 해설

- ① A의 응답비는 3.00이 맞지만 C의 4.00보다 작다.
- ② B는 2.50으로 가장 작은 축이다. 대기시간만 보면 두 번째지만 서비스시간도 10이라 상쇄된다.
- ③ 프로세스는 맞다. 3.00은 응답비를 **대기시간 ÷ 서비스시간**($9 \div 3$)으로 구한 값이다. 분자에 서비스시간을 더해야 한다.
- ④ (정답) $(9 + 3) \div 3 = 4.00$ 으로 가장 크다.
- ⑤ D는 가장 오래(20) 기다렸지만 서비스시간도 20이라 응답비가 2.00으로 가장 작다. HRN은 대기시간만 보지 않는다.

04 ④ 페이지교체

새 페이지 P9를 들일 때 교체되는 페이지와, 그 뒤 포인터가 가리키는 프레임은?

포인터를 시계처럼 돌리며 참조비트를 본다.

차례	프레임	페이지	참조비트	처리
1	0	P1	1	0으로 지우고 통과
2	1	P2	1	0으로 지우고 통과
3	2	P3	0	교체

P3가 나가고 그 자리에 P9가 들어간다. 포인터는 교체한 자리에 머무르지 않고 다음 프레임인 3으로 옮겨 간다.

참조비트가 1이라고 해서 영원히 남지는 않는다. 한 번 봐주고 지워 두므로, 다음 바퀴에 다시 참조되지 않았다면 그때는 교체된다. 이것이 「2차 기회」라는 이름의 뜻이다.

선택지 해설

- ① P1은 참조비트가 1이라 이번에는 살아남는다. 참조비트를 보지 않으면 FIFO가 되어 가장 오래된 P1이 나간다고 보게 된다.
- ② P2도 참조비트가 1이다. 통과하며 0으로 지워질 뿐이다.
- ③ 교체되는 페이지는 맞다. 포인터는 **교체한 자리**가 아니라 그 다음 자리로 옮긴다.
- ④ (정답) 참조비트 1인 P1·P2를 지우며 지나고, 0인 P3를 교체한다. 포인터는 프레임 3으로 간다.
- ⑤ P4는 아직 보지도 않았다. 포인터가 프레임 2에서 멈췄다.

먼저 가용 자원과 각 프로세스의 잔여 요구를 구한다.

할당 합계는 $A\ 7 \cdot B\ 5 \cdot C\ 7$ 이므로 **가용 = (2, 1, 1)** 이다.

프로세스	잔여 요구 (Max - 할당)	가용 (2,1,1)로 되나
P1	(3, 2, 1)	A가 $3 > 2$ — 안 된다
P2	(1, 1, 0)	된다
P3	(4, 2, 4)	안 된다
P4	(1, 1, 2)	C가 $2 > 1$ — 안 된다

P2만 시작할 수 있다. P2가 끝나며 자기 할당 (2,1,2)를 돌려주므로 가용은 (4, 2, 3)이 된다.

이제 P1(3,2,1)이 된다. P1이 끝나면 가용 (5,3,5), P4(1,1,2)가 되고 그 뒤 가용 (6,4,7)에서 P3(4,2,4)가 된다.

안전 순서는 **P2 → P1 → P4 → P3** 이다.

선택지 해설

- ① P1은 처음부터 못 움직인다. 잔여 요구 A 3이 가용 A 2를 넘는다.
- ② P2 뒤 가용은 (4,2,3)인데 P3는 C를 4개 더 필요로 한다. 3개뿐이다.
- ③ P3의 잔여 요구는 (4,2,4)로 이 상태에서 가장 크다. 맨 앞에 설 수 없다.
- ④ P4는 C가 2 필요한데 가용 C가 1이다. 할당량이 P2와 비슷해 보여도 ****남은 요구****가 다르다.
- ⑤ (정답) 가용 (2,1,1)에서 **P2만 되고, 그 뒤 (4,2,3)에서 P1이 된다.**

06 ② 동기화

모든 연산이 끝난 뒤 S의 값과 대기 중인 프로세스 수는?

이 정의에서 S가 음수이면 **그 절댓값이 곧 대기 중인 프로세스 수**다.

연산	P	P	P	P	P	V	V	P
S	2	1	0	-1	-2	-1	0	-1
대기	0	0	0	1	2	1	0	1

P가 6번, V가 2번이므로 $3 - 6 + 2 = -1$ 이다. 대기 중인 프로세스는 1개다.

선택지 해설

- ① -2는 다섯 번째 P 직후의 값이다. 그 뒤 V가 두 번 있었다.
- ② (정답) $3 - 6 + 2 = -1$ 이고, **절댓값이 대기 수이므로 1개다.**
- ③ 연산 횟수만 세어도 P가 V보다 4번 많다. 0이 될 수 없다.
- ④ S를 ****0 아래로 내리지 않는**** 정의를 쓰면 이 값이 나온다. 그 정의에서는 대기 수를 따로 세야 하지만, 여기서는 음수를 허용한다.
- ⑤ 마지막 P 하나만 대기한다. V 두 번이 앞의 두 프로세스를 이미 깨웠다.

07 ③ 정규화

이 릴레이션이 만족하는 가장 높은 정규형과, BCNF를 위배하는 함수 종속은?

먼저 후보키를 구한다. 폐포를 하나씩 잡아 보면

속성 집합	폐포	후보키인가
AB	$AB \rightarrow C \rightarrow D \dots ABCD$	○
AC	$AC \rightarrow D(C \rightarrow D) \rightarrow B(D \rightarrow B) \dots ABCD$	○
AD	$AD \rightarrow B(D \rightarrow B) \rightarrow C(AB \rightarrow C) \dots ABCD$	○
$A \cdot BC \cdot BD$	전체가 되지 않는다	×

후보키가 AB · AC · AD 이므로 **A · B · C · D 네 속성이 모두 기본속성(prime)**이다.

제3정규형은 「모든 종속의 오른쪽이 기본속성이거나, 왼쪽이 슈퍼키」를 요구한다. C → D 의 D도, D → B 의 B도 기본속성이므로 **3NF는 만족한다**.

BCNF는 더 엄하다. **왼쪽이 슈퍼키여야만** 한다. C의 폐포는 CDB로 A가 빠지고, D의 폐포는 DB뿐이다. 둘 다 슈퍼키가 아니므로 **C → D 와 D → B 가 BCNF를 위배한다**.

선택지 해설

- ① AB는 후보키이므로 $AB \rightarrow C$ 는 어느 정규형도 위배하지 않는다.
- ② 위배 종속은 맞지만 정규형이 틀렸다. 오른쪽 D·B가 모두 기본속성이라 3NF까지는 올라간다.
- ③ (정답) 3NF는 만족하고, 왼쪽이 슈퍼키가 아닌 **C → D · D → B 가 BCNF를 위배한다**.
- ④ D → B 도 함께 위배한다. D의 폐포는 DB로 ABCD가 되지 못한다.
- ⑤ BCNF라면 위배 종속이 없어야 한다. C의 폐포에 A가 없다.

08 ⑤ 트랜잭션

이 스케줄에 대한 판정으로 옳은 것은?

충돌하는 짝을 찾아 선행 그래프의 화살표를 그린다.

앞선 연산	뒤선 연산	자료	화살표
T1 read(A) · 1	T2 write(A) · 5	A	T1 → T2
T1 read(A) · 1	T3 write(A) · 6	A	T1 → T3
T1 write(B) · 3	T3 read(B) · 4	B	T1 → T3
T2 read(A) · 2	T3 write(A) · 6	A	T2 → T3
T2 write(A) · 5	T3 write(A) · 6	A	T2 → T3

화살표는 T1 → T2, T1 → T3, T2 → T3 뿐이다. **순환이 없으므로 충돌 직렬가능하다**.

들어오는 화살표가 없는 것은 T1, T1을 뺀 뒤 없는 것은 T2다. 순서는 **T1 → T2 → T3 하나로 정해진다**.

선택지 해설

- ① 선행 그래프에 순환이 없다. T3에서 나가는 화살표가 하나도 없다.
- ② T1 read(A)가 T2 write(A)보다 앞서므로 T1이 먼저다.
- ③ T3은 T1의 write(B)를 읽었다. T1보다 앞설 수 없다.
- ④ 세 화살표가 세 트랜잭션을 한 줄로 켜다. 위상 순서는 하나뿐이다.
- ⑤ (정답) **T1 → T2 → T3**.

조건을 어디에 두느냐가 결과를 바꾼다.

㉠ 은 조건이 ON에 있다. 조건은 **짜을 맺을 때만** 쓰이고, 짜을 못 맺은 왼쪽 행은 그대로 NULL을 달고 남는다.

부서	㉠ 결과	㉡ 결과
D1 영업	김대리(300) — 1행	김대리 — 1행
D2 개발	박과장·최사원 — 2행	박과장·최사원 — 2행
D3 총무	NULL — 1행	걸러짐
D4 인사	NULL — 1행	걸러짐
합	5행	3행

㉡ 은 조인을 다 한 뒤 WHERE로 거른다. 짜을 못 맺은 D3·D4는 급여가 NULL이라 $NULL \geq 300$ 이 참이 되지 못하고 사라진다. **WHERE에 오른쪽 표의 조건을 쓰면 외부조인이 사실상 내부조인이 된다.**

선택지 해설

- ① ㉠ 이 3행이 되려면 조건이 ON이 아니라 WHERE에 있어야 한다.
- ② 두 값이 뒤바뀌었다. 행이 더 많이 남는 쪽은 조건을 ON에 둔 ㉠ 이다.
- ③ ㉡ 의 D3·D4는 급여가 NULL이다. NULL과의 비교는 참이 되지 않아 걸러진다.
- ④ (정답) ㉠ 은 D3·D4가 NULL 행으로 남아 5행, ㉡ 은 그 둘이 걸려져 3행이다.
- ⑤ 6행은 조건을 아예 걸지 않은 왼쪽 외부조인의 행 수다(D1 2행 + D2 2행 + D3 1행 + D4 1행).

노드가 70% 차 있으므로 실제로 담기는 것은 $100 \times 0.7 = 70$ 개다.

리프 노드 수 = $350,000 \div 70 = 5,000$ 개

레벨	노드 수	셈
1 (리프)	5,000	$350,000 \div 70$
2	72	$5,000 \div 70 \rightarrow$ 올림
3	2	$72 \div 70 \rightarrow$ 올림
4 (루트)	1	$2 \div 70 \rightarrow$ 올림

레벨이 4개이므로 **높이는 4**다. 레벨 3에 노드가 2개만 있어도 그 둘을 묶을 루트가 필요하다.

선택지 해설

- ① 3,500개는 노드를 100% 채운다고 본 값이다($350,000 \div 100$). 높이 3도 그 가정에서 나온다.
- ② 리프 수가 3,500이면 그 위는 $35 \rightarrow 1$ 이라 높이가 3이다. 두 값이 서로 맞지 않는다.
- ③ 레벨 3에 노드가 2개 남는다. 루트가 하나 더 필요하다.
- ④ (정답) 리프 5,000개, 레벨 4개.
- ⑤ 레벨 4에서 이미 노드가 1개(루트)다. 더 올라갈 곳이 없다.

세 함수가 NULL을 저마다 다르게 다룬다.

함수	세는 것	값
COUNT(*)	행을 센다. NULL도 행이다	6
COUNT(성과급)	그 열의 NULL 아닌 값을 센다	3
AVG(성과급)	NULL을 아예 빼고 평균	$600 \div 3 = 200$

AVG는 NULL을 0으로 보지 않는다. 분모도 3이지 6이 아니다. 「모르는 값」과 「0」은 다른 것이라 평균에 끌어들이지 않는다.

선택지 해설

- ① COUNT(*)는 6이다. NULL이 든 행도 행으로 센다.
- ② (정답) $6 \cdot 3 \cdot 200$. AVG의 분모는 NULL을 뺀 3이다.
- ③ 100은 AVG를 $600 \div 6$ 으로 구한 값이다. NULL을 0으로 보고 분모에 넣었다.
- ④ 600은 AVG가 아니라 SUM(성과급)의 값이다.
- ⑤ COUNT(성과급)은 6이 될 수 없다. 값이 있는 행은 셋뿐이다.

폐포를 구해 전체 속성이 나오는지 본다.

집합	폐포를 넓히는 과정	결과
A	$A \rightarrow BC \rightarrow D(B \rightarrow D) \rightarrow E(CD \rightarrow E)$	ABCDE ○
E	$E \rightarrow A \rightarrow BC \rightarrow D \rightarrow E$	ABCDE ○
BC	$BC \rightarrow D(B \rightarrow D) \rightarrow E(CD \rightarrow E) \rightarrow A(E \rightarrow A)$	ABCDE ○
CD	$CD \rightarrow E \rightarrow A \rightarrow BC$	ABCDE ○
B	$B \rightarrow D$. 여기서 멈춘다	BD ×
$C \cdot D \cdot BD$	넓어지지 않는다	×

후보키는 $A \cdot E \cdot BC \cdot CD$ 넷이다. A나 E를 품은 두 속성 집합($AB \cdot CE$...)은 더 작은 후보키를 안고 있으므로 최소성을 잃어 후보키가 아니다.

선택지 해설

- ① BC와 CD도 후보키다. 둘 다 폐포가 ABCDE까지 넓어진다.
- ② CD가 빠졌다. $CD \rightarrow E \rightarrow A \rightarrow BC$ 로 전체가 된다.
- ③ BC가 빠졌다. $B \rightarrow D$ 를 쓰면 CD를 얻고, 거기서 E와 A로 이어진다.
- ④ (정답) $A \cdot E \cdot BC \cdot CD$ 넷이다.
- ⑤ B는 후보키가 아니다. B의 폐포는 BD에서 멈춘다 — C가 없으면 $CD \rightarrow E$ 를 쓸 수 없다.

먼저 자연조인 $R \bowtie S$ 를 B로 맞춘다.

R의 행	맞는 S의 행	나오는 행 수
(1, x)	(x, p) · (x, q)	2
(1, y)	(y, p)	1
(2, x)	(x, p) · (x, q)	2
(3, z)	없다	0
합		5

여기서 A만 뽑으면 1, 1, 1, 2, 2 다.

관계대수의 릴레이션은 집합이다. π 는 중복을 없애므로 ㉠ 은 {1, 2} — 2행이다.

SQL의 테이블은 다중집합이다. SELECT는 DISTINCT를 붙이지 않는 한 중복을 그대로 두므로 ㉡ 은 5행이다.

선택지 해설

- ① ㉡ 에 DISTINCT가 없다. SQL은 중복을 지우지 않는다.
 ② (정답) 관계대수의 π 는 집합 연산이라 2행, SQL은 다중집합이라 5행이다.
 ③ A에 남은 값은 1과 2뿐이다. (3, z)는 짝이 없어 조인에서 빠졌다.
 ④ 두 값이 뒤바뀌었다. 중복을 지우는 쪽은 관계대수다.
 ⑤ ㉠ 이 5행이 되려면 관계대수가 다중집합을 다뤄야 한다.

부질의가 내놓는 관리자사번 집합은 {NULL, 1, 1, 2} 다.

㉠ 사번 NOT IN (NULL, 1, 2) 는 사번 <> NULL AND 사번 <> 1 AND 사번 <> 2 로 풀린다. 사번 <> NULL 은 참도 거짓도 아닌 UNKNOWN이라 AND 로 묶인 전체가 결코 참이 되지 못한다. WHERE는 참인 행만 남기므로 0행이다.

사번	㉠ NOT IN	㉡ NOT EXISTS
1 김부장	UNKNOWN	이대리·박과장이 있다 → 거짓
2 이대리	UNKNOWN	최사원이 있다 → 거짓
3 박과장	UNKNOWN	없다 → 참
4 최사원	UNKNOWN	없다 → 참

㉡ 은 행을 하나씩 대조하는 상관 부질의다. m.관리자사번 = e.사번 은 NULL 행에서 그냥 맞지 않을 뿐이라 결과를 어지럽히지 않는다. 2행이다.

선택지 해설

- ① (정답) ㉠ 은 NULL 때문에 0행, ㉡ 은 박과장·최사원 2행이다.
 ② ㉡ 은 NULL에 걸리지 않는다. 부질의가 「맞는 행이 있느냐」만 보기 때문이다.
 ③ 두 값이 뒤바뀌었다. NULL에 무너지는 쪽은 NOT IN 이다.
 ④ 부질의 결과에 NULL이 없었다면 ㉠ 도 2행이 됐을 것이다. 여기서는 김부장의 관리자사번이 NULL이다.
 ⑤ 4행은 조건을 아예 걸지 않은 행 수다.

데 이 터 통 신

15 ② 프레임

이 데이터를 보낼 때 삽입되는 비트 수와, 실제로 전송되는 비트 수는?

왼쪽부터 세면서 1이 다섯 개 이어질 때마다 0을 끼운다. 끼우고 나면 이어진 개수를 0부터 다시 센다.

자리	비트	이어진 1	처리
1	0	0	
2~6	1 1 1 1 1	1→5	0 삽입
7~8	1 1	1→2	
9	0	0	
10~14	1 1 1 1 1	1→5	0 삽입
15	1	1	
16~17	0 0	0	
18~22	1 1 1 1 1	1→5	0 삽입

삽입 3개, 전송 $22 + 3 = 25$ 비트다.

받는 쪽은 반대로 한다 — 1이 다섯 개 이어진 바로 뒤의 0은 데이터가 아니므로 떼어 낸다.

선택지 해설

- ① 마지막 다섯 개(18~22번째)를 놓친 값이다. 프레임 끝에 붙은 것도 세어야 한다.
- ② (정답) 6·14·22번째 자리에서 하나씩, 모두 3개를 끼워 25비트가 된다.
- ③ 삽입 개수는 맞지만 전송 비트 수가 원래 길이 그대로다. 끼운 만큼 늘어난다.
- ④ 7~8번째의 1 두 개까지 이어 세면 4개가 나온다. 0을 끼운 뒤에는 **이어진 개수를 다시 센다.**
- ⑤ 1이 나올 때마다 세는 것이 아니라 **다섯 개가 이어질 때만** 끼운다.

16 ④ 변조·부호화

이 회선으로 낼 수 있는 최대 비트율은?

두 단계로 나눠 본다.

- ① 초당 몇 번 신호를 바꿀 수 있나 — 나이퀴스트에 따라 최대 보율은 대역폭의 2배다. $2 \times 3,000 = 6,000$ baud
- ② 신호 한 번에 몇 비트를 실나 — 64-QAM은 서로 다른 신호 상태가 64가지다. $\log_2 64 = 6$ 비트

$$6,000 \times 6 = 36,000\text{bps}$$

보율(baud)은 초당 신호 변화 횟수이고 비트율(bps)은 초당 비트 수다. 둘은 심볼당 비트 수만큼 차이 난다. 64-QAM에서는 6배다.

선택지 해설

- ① 6,000은 보율이다. 아직 심볼당 비트 수를 곱하지 않았다.
- ② 18,000은 대역폭의 2배를 취하지 않고 $3,000 \times 6$ 으로 구한 값이다.
- ③ 24,000은 심볼당 비트 수를 4로 본 값이다. 그것은 16-QAM이다. 64-QAM은 $\log_2 64 = 6$ 이다.
- ④ (정답) $2 \times 3,000 \times 6 = 36,000\text{bps}$.
- ⑤ 384,000은 64를 그대로 곱한 값이다($6,000 \times 64$). 상태 수가 아니라 **그 로그**가 비트 수다.

네 트 워 크

17 ① 데이터링크

플러딩되는 프레임의 개수와, 네 프레임이 모두 지나간 뒤 MAC 주소 테이블에 남는 항목 수는?

스위치는 프레임이 들어올 때마다 보내는 쪽 주소를 배우고, 받는 쪽 주소를 찾아본다. 모르면 들어온 포트를 뺀 모든 포트로 내보낸다(플러딩).

차례	배우는 것	목적지를 아나	처리
1 A→B	A = 포트1	B를 모른다	플러딩
2 B→A	B = 포트2	A = 포트1	포트1로만
3 C→A	C = 포트3	A = 포트1	포트1로만
4 D→C	D = 포트4	C = 포트3	포트3으로만

플러딩은 첫 프레임 한 번뿐이다. A가 B에게 보내는 순간 스위치는 A만 알 뿐 B가 어디 있는지 모른다. 그런데 B가 답장을 보내면서 자기 자리를 알려 주므로, 그 뒤로는 찾을 필요가 없다.

네 호스트가 모두 한 번씩 보냈으므로 테이블에는 4개가 남는다.

선택지 해설

- ① (정답) 플러딩은 첫 프레임 하나뿐이고, 테이블에는 A·B·C·D 4개가 남는다.
 ② 두 번째 프레임(B→A)은 플러딩되지 않는다. 첫 프레임이 이미 A의 자리를 알려 주었다.
 ③ 넷이 모두 한 번씩 **보내는 쪽**이 되었다. 항목은 4개다.
 ④ 세 번째(C→A)도 목적지 A를 안다. 스위치는 목적지를 알면 그 포트로만 보낸다.
 ⑤ 스위치는 배운 것을 잊지 않는다(수명이 다하기 전까지). 모든 프레임을 플러딩하는 것은 허브다.

18 ② 서브네팅

인사팀에 지정되는 네트워크 주소와, 어느 팀에도 지정되지 않고 남는 주소의 개수는?

필요한 호스트 수를 담을 수 있는 가장 작은 크기를 고른다. 쓸 수 있는 호스트는 (주소 수 - 2) 다.

팀	필요	프리픽스	주소 수	쓸 수 있는 호스트	구간
영업	100	/25	128	126	.0 ~ .127
개발	50	/26	64	62	.128 ~ .191
총무	25	/27	32	30	.192 ~ .223
인사	10	/28	16	14	.224 ~ .239

인사팀은 192.168.100.224/28 이다.

.240 부터 .255 까지가 남는다. 모두 16개다.

큰 것부터 자르는 이유는 조각이 어긋나지 않게 하려는 것이다. 작은 것부터 자르면 /25 를 놓을 자리가 중간에 걸려 못 쓰게 된다.

선택지 해설

- ① .192는 총무팀(/27)의 자리다. 인사팀은 그 다음이다.
 ② (정답) 인사팀은 .224/28, 남는 주소는 .240 ~ .255 로 16개다.
 ③ 14는 남는 **주소 수**가 아니라 /28 에서 쓸 수 있는 **호스트 수**다. .240 이후는 배정 자체가 없으므로 네트워크·브로드캐스트를 뺀 이유가 없다.
 ④ /29 는 주소가 8개뿐이라 쓸 수 있는 호스트가 6대다. 10대를 담지 못한다.
 ⑤ .240/28 은 남는 자리다. 인사팀은 그 앞 칸을 받는다.

송신 윈도우는 두 윈도우의 **작은 쪽**이다.

$\min(12,000, 8,000) = 8,000\text{바이트}$

그 가운데 5,000바이트는 이미 나가서 확인을 기다리는 중이다.

$8,000 - 5,000 = 3,000\text{바이트}$

두 윈도우는 서로 다른 것을 막는다. cwnd는 **망이 밀리는 것을**, rwnd는 **받는 쪽 버퍼가 넘치는 것을** 막는다. 둘 중 하나라도 넘으면 안 되므로 작은 쪽을 따른다.

선택지 해설

① (정답) $\min(12,000, 8,000) - 5,000 = 3,000\text{바이트}$.

② 4,000은 두 윈도우의 차($12,000 - 8,000$)다. 윈도우는 빼는 것이 아니라 작은 쪽을 고르는 것이다.

③ 7,000은 큰 쪽인 cwnd에서 미확인분을 뺀 값이다. 수신 측 버퍼를 넘긴다.

④ 8,000은 송신 윈도우 자체다. 미확인 5,000바이트가 이미 그 안을 쓰고 있다.

⑤ 15,000은 두 윈도우를 더한 값이다. 두 제약은 더해지지 않는다.

계층마다 자기 헤더를 붙이며 내려간다. 이것을 캡슐화라고 한다.

계층	붙는 것	결과 이름
전송	포트 번호	세그먼트(TCP)
네트워크	IP 주소	패킷
데이터링크	MAC 주소 · 뒤에 FCS	프레임
물리	아무 주소도 붙지 않는다	비트

물리 계층은 프레임을 **전기·빛 신호로 바꿔 내보낼 뿐**이다. 여기에는 헤더도 주소도 없다. 포트 번호는 전송 계층에서 이미 한 번 붙었고 다시 붙을 일이 없다.

데이터링크 계층만 **앞뒤로** 붙인다. 앞에는 헤더(MAC 주소 등), 뒤에는 트레일러(FCS)다.

선택지 해설

① 맞다. 포트 번호는 전송 계층의 몫이고 결과는 세그먼트다.

② 맞다. IP 주소는 네트워크 계층에서 붙고 결과는 패킷이다.

③ 맞다. MAC 주소는 데이터링크 계층에서 붙고 결과는 프레임이다.

④ 맞다. 데이터링크 계층은 앞에 헤더, 뒤에 FCS를 붙인다.

⑤ (정답) 틀렸다. 물리 계층은 주소를 붙이지 않는다. 프레임을 신호로 바꿔 보낼 뿐이다.

A에서 출발해 값이 확정되는 차례로 적으면 이렇다.

노드	최소 비용	경로
B	2	A-B
C	3	A-B-C (2+1). 직통 A-C 5보다 싸다
D	6	A-B-C-D (3+3)
E	8	A-B-C-D-E (6+2)
F	9	A-B-C-D-E-F (8+1)

D-E가 끊기면 E로 가는 값싼 길이 사라진다. F로 가는 길은 둘만 남는다.

$$A-B-C-D-F = 3 + 3 + 6 = 12$$

$$A-B-C-E-F = 3 + 8 + 1 = 12$$

둘 다 12라 최소 비용은 12다. 링크 하나가 빠지면서 3만큼 비싸졌다.

선택지 해설

- ① 끊긴 뒤에도 9가 나오려면 D-E를 지나야 한다. 그 링크가 없다.
- ② 11은 A에서 E까지의 값이다(A-B-C-E = 3+8). F까지 1이 더 든다.
- ③ (정답) 9와 12.
- ④ 10은 어느 경로로도 나오지 않는다. A-C를 직통 5로 잡으면 5+3+6 = 14로 오히려 늘어난다.
- ⑤ 12는 D-E를 쓰지 않은 값이다. 링크가 살아 있을 때는 그보다 싼 9가 있다.

네 메시지의 앞 글자를 따 DORA라고 부른다.

차례	메시지	보내는 쪽	하는 일
1	DISCOVER	단말	「서버 있습니까」 — 브로드캐스트
2	OFFER	서버	「이 주소를 쓰시겠습니까」
3	REQUEST	단말	「그것으로 하겠습니다」 — 역시 브로드캐스트
4	ACK	서버	임대 확정

단말이 아직 주소가 없으므로 DISCOVER는 브로드캐스트로 나간다. REQUEST도 브로드캐스트인데, 여러 서버가 OFFER를 보냈을 때 고르지 않은 서버들에게도 알려 주소를 풀어 주게 하려는 것이다.

포트는 서버 67 · 단말 68이다(UDP).

선택지 해설

- ① (정답) DISCOVER → OFFER → REQUEST → ACK, 서버 포트는 67번이다.
- ② 68번은 단말이 듣는 포트다. 서버가 ACK를 보낼 때 목적지로 쓴다.
- ③ OFFER를 받아야 무엇을 REQUEST할지 정해진다. 순서가 뒤집혔다.
- ④ 서버는 DISCOVER를 받고 나서야 OFFER를 보낸다. 먼저 말을 걸지 않는다.
- ⑤ 주소를 모르는 단말이 REQUEST부터 보낼 수는 없다.

먼저 프레임 하나를 회선에 밀어 넣는 데 걸리는 시간을 구한다.

$$10,000\text{비트} \div 10,000,000\text{bps} = 0.001\text{초} = 1\text{ms}$$

첫 프레임을 보내기 시작해 그 확인응답이 돌아올 때까지는

$$1\text{ms}(\text{전송}) + 10\text{ms}(\text{가는 길}) + 10\text{ms}(\text{오는 길}) = 21\text{ms}$$

그 21ms 동안 원도 크기만큼, 곧 4개를 연달아 보낼 수 있다. 실제로 회선을 쓴 시간은 $4 \times 1 = 4\text{ms}$ 다.

$$4 \div 21 = 0.1904... \rightarrow \text{약 } 19.0\%$$

회선을 가득 채우려면 원도가 21개는 되어야 한다. **거리가 멀수록 원도를 키워야 하는 이유**가 이것이다.

선택지 해설

- ① 4.8%는 원도가 1일 때, 곧 정지-대기 방식의 이용률이다($1 \div 21$).
- ② (정답) $4 \times 1\text{ms} \div 21\text{ms} \approx 19.0\%$.
- ③ 20.0%는 분모에서 전송 시간 1ms를 빼고 왕복 20ms만 쓴 값이다. 첫 프레임을 밀어 넣는 시간도 기다림에 든다.
- ④ 40.0%는 전파지연을 편도 5ms로 본 값이다. 왕복이므로 두 번 든다.
- ⑤ 100%가 되려면 확인응답이 돌아오기 전에 원도를 다 못 쓰는 일이 없어야 한다. 원도가 21개는 되어야 한다.

실을 수 있는 데이터는 MTU에서 헤더를 뺀 만큼이다.

$$1,500 - 20 = 1,480\text{바이트. } 1,480\text{은 } 8\text{의 배수}(8 \times 185)\text{라 그대로 쓴다.}$$

$$\text{나를 데이터는 } 4,000 - 20 = 3,980\text{바이트다.}$$

조각	데이터	원본에서의 자리	오프셋
1	1,480	0 ~ 1,479	$0 \div 8 = 0$
2	1,480	1,480 ~ 2,959	$1,480 \div 8 = 185$
3	1,020	2,960 ~ 3,979	$2,960 \div 8 = 370$

조각은 **3개**, 마지막 오프셋은 **370**이다.

헤더 20바이트는 **조각마다 새로 붙는다**. 그래서 마지막 조각의 전체 길이는 $20 + 1,020 = 1,040\text{바이트}$ 가 된다.

선택지 해설

- ① 185는 ****두 번째**** 조각의 오프셋이다.
- ② (정답) **3개, 마지막 오프셋 370**.
- ③ 2,960은 바이트 단위 자리다. 오프셋 필드에는 8로 나눈 값을 적는다.
- ④ 조각은 3개다. 4,000을 1,500으로 나눠 셋에 나머지가 있다고 보면 4개가 되지만, 나르는 것은 헤더를 뺀 3,980이다.
- ⑤ 500은 $4,000 \div 8$ 로, 원본 전체를 8로 나눈 값이다. 오프셋은 ****데이터 부분****의 자리를 가리킨다.

25 ① 테스트

문장 커버리지 100%와 분기 커버리지 100%를 채우는 데 필요한 최소 테스트 케이스 수를 바르게 짚은 것은?

문장 커버리지는 모든 문장을 한 번씩 지나면 된다. $a > 0$ 이고 $b > 0$ 인 케이스 하나면 $x=0 \cdot x=1 \cdot x=x+1 \cdot \text{print}$ 를 모두 지난다. 1개다.

분기 커버리지는 각 판단의 참과 거짓을 모두 지나야 한다. else 가 없어도 조건이 거짓일 때 건너뛰는 길이 하나의 분기다.

케이스	$a > 0$	$b > 0$	지나는 분기
$a=1, b=1$	참	참	참·참
$a=0, b=0$	거짓	거짓	거짓·거짓

두 케이스로 네 분기를 다 덮으므로 2개다.

분기가 넷이라고 케이스가 넷 필요한 것은 아니다. 한 케이스가 여러 분기를 한꺼번에 지난다. 네 가지 조합을 다 시험하라는 것은 조건 조합 커버리지이고, 그것은 더 센 기준이다.

선택지 해설

- ① (정답) 문장 1개 · 분기 2개.
- ② 분기 1개로는 거짓 갈래를 지나지 못한다. 조건이 거짓일 때 건너뛰는 길도 분기다.
- ③ 문장은 한 케이스면 된다. a와 b가 모두 양수인 값 하나로 네 문장을 다 지난다.
- ④ 4는 두 조건의 참·거짓 조합 수다. 그것은 조건 조합 커버리지가 요구하는 값이다.
- ⑤ 문장 4개는 문장의 **개수**이지 케이스 수가 아니다.

26 ⑤ 설계

UML 다이어그램과 그 쓰임을 짚은 것으로 옳지 않은 것은?

UML 다이어그램은 구조를 그리는 것과 행위를 그리는 것으로 갈린다.

갈래	다이어그램	무엇을 그리나
구조	클래스	클래스와 그 사이의 관계
	배치(deployment)	어떤 하드웨어 노드에 무엇을 올리는지
행위	유스케이스	액터가 하는 일
	순차	시간 순서의 메시지
	상태 · 활동	상태 전이 · 처리 흐름

처리 흐름의 분기와 병렬을 그리는 것은 **활동(activity) 다이어그램**이다. 배치 다이어그램은 노드와 산출물을 그리는 구조 다이어그램이라 흐름을 담지 않는다.

선택지 해설

- ① 맞다. 클래스 다이어그램은 대표적인 구조 다이어그램이다.
- ② 맞다. 요구 단계에서 시스템의 범위를 잡을 때 쓴다.
- ③ 맞다. 생명선을 세로로 두고 메시지를 가로 화살표로 그린다.
- ④ 맞다. 사건과 상태 전이를 그린다.
- ⑤ (정답) 틀렸다. 이 설명은 ****활동 다이어그램****의 것이다. 배치 다이어그램은 하드웨어 노드에 무엇을 올리든지 그린다.

먼저 작업마다 기대 시간을 구한다.

작업	식	기대 시간
A	$(2 + 16 + 12) \div 6$	5
B	$(3 + 24 + 9) \div 6$	6
C	$(1 + 16 + 7) \div 6$	4
D	$(4 + 28 + 16) \div 6$	8

C는 A와 B가 모두 끝나야 시작한다. 둘은 나란히 도니 늦게 끝나는 쪽을 기다린다 — $\max(5, 6) = 6$.

$$6 + 4 + 8 = 18\text{일}$$

임계경로는 B → C → D 다. A는 5일이면 끝나 1일의 여유가 있다 — A가 하루 늦어도 전체 기간은 그대로다.

선택지 해설

- ① 15일은 A → C → D 로 잡은 값이다($5 + 4 + 8$). C는 B도 기다려야 한다.
- ② 17일은 정상 시간만으로 계산한 값이다($6 + 4 + 7$). PERT는 낙관·비관까지 가중해 넣는다.
- ③ (정답) $6 + 4 + 8 = 18\text{일}$.
- ④ 19일은 기대 시간을 (낙관 + 정상 + 비관) ÷ 3 으로 구한 값이다. 정상값에는 4를 곱하고 6으로 나눈다.
- ⑤ 23일은 네 작업을 모두 더한 값이다($5 + 6 + 4 + 8$). A와 B는 동시에 돌아 더해지지 않는다.

스텝은 아직 없는 아래쪽을 대신하는 껍데기이고, 드라이버는 아직 없는 위쪽을 대신해 불러 주는 것이다.

하향식은 M1부터 시작한다. M1을 뺀 모든 모듈이 연젠가 한 번은 스텝으로 대신되므로 $7 - 1 = 6\text{개}$ 다.

불일 차례	스텝이 필요한 자리
M1 단독	M2 · M3 · M4
M2 불임	M5 · M6
M4 불임	M7

상향식은 앞(M3 · M5 · M6 · M7)부터 시작한다. 그것들을 불러 줄 **앞이 아닌 모듈** 자리에 드라이버가 필요하다 — M1 · M2 · M4 로 3개다.

앞 모듈은 아래에 아무것도 없으므로 스텝이 필요 없고, 최상위 모듈은 위에 아무것도 없으므로 드라이버가 필요 없다.

선택지 해설

- ① 스텝 3개는 M1을 시험할 때 **한 번에** 필요한 수다. 그 아래를 붙여 나가며 더 필요해진다.
- ② 두 값이 모두 어긋났다.
- ③ (정답) 스텝 6개(M1을 뺀 전부) · 드라이버 3개(M1 · M2 · M4).
- ④ 드라이버는 앞이 아닌 모듈에만 필요하다. M3는 앞이므로 그것을 위한 드라이버는 M1의 것과 같다.
- ⑤ 7은 모듈 전체 수다. 최상위에는 드라이버가, 앞에는 스텝이 필요 없다.

29 ① 포인터

다음 C 프로그램의 출력은?

포인터에 정수를 더하면 바이트가 아니라 칸이 움직인다. $p + 2$ 는 주소로는 8바이트 뒤이지만 가리키는 것은 $a[2]$ 다.

식	가리키는 원소	값
$*p$	$a[2]$	30
$*(p - 1)$	$a[1]$	20
$p[1]$	$a[3]$	40

$p[1]$ 은 $*(p + 1)$ 과 같은 뜻이다. 첨자는 배열 이름이 아니라 지금 그 포인터가 선 자리에서 센다. p 가 $a[2]$ 에 서 있으므로 $p[1]$ 은 $a[3]$ 이다.

$20 + 40 = 60$ 이므로 출력은 30 60 이다.

선택지 해설

- ① (정답) 30 과 $20 + 40 = 60$.
- ② 20은 p 를 한 칸만 옮긴 값($a[1]$)이다. 2를 더했다.
- ③ 50은 $20 + 30$ 으로, $p[1]$ 을 p 자신($a[2]$)으로 본 값이다.
- ④ 70은 $30 + 40$ 으로, $*(p - 1)$ 을 $*p$ 로 본 값이다. 빼면 앞 칸으로 간다.
- ⑤ 40은 첨자를 1부터 센 값이다. C의 배열 첨자는 0부터다.

30 ③ 배열

다음 프로그램이 출력하는 sum의 값은?

먼저 배열이 이렇게 채워진다.

	j=0	j=1	j=2	j=3
i=0	0	1	2	3
i=1	4	5	6	7
i=2	8	9	10	11

둘째 반복은 $a[i][3 - i]$ 를 더한다. i 가 커질수록 열이 왼쪽으로 옮겨 가는 부대각선이다.

$a[0][3] + a[1][2] + a[2][1] = 3 + 6 + 9 = 18$

선택지 해설

- ① 12는 첫 열의 합이다($0 + 4 + 8$).
- ② 15는 주대각선 $a[i][i]$ 의 합이다($0 + 5 + 10$). 여기서는 열 첨자가 $3 - i$ 다.
- ③ (정답) $3 + 6 + 9 = 18$.
- ④ 21은 마지막 열의 합이다($3 + 7 + 11$). 열이 3에 붙박여 있다고 본 값이다.
- ⑤ 66은 열두 원소를 모두 더한 값이다(0부터 11까지).

밑에서부터 쌓아 올린다. n이 0이든 1이든 1을 돌려준다.

n	0	1	2	3	4	5	6
f(n)	1	1	3	5	9	15	25

$$f(2) = 1 + 1 + 1 = 3$$

$$f(3) = 3 + 1 + 1 = 5$$

$$f(4) = 5 + 3 + 1 = 9$$

$$f(5) = 9 + 5 + 1 = 15$$

$$f(6) = 15 + 9 + 1 = 25$$

피보나치와 모양이 같지만 **부를 때마다 1이 더 붙는다**. 그 1이 쌓여 n이 커질수록 차이가 벌어진다.

선택지 해설

- ① 13은 + 1 을 빼고 계산한 값, 곧 피보나치 수열의 여섯째 항이다.
- ② 15는 f(5)의 값이다. 한 단계 덜 갔다.
- ③ 20은 f(0)을 0으로 본 값이다. 이 함수는 $n \leq 1$ 일 때 1을 돌려준다.
- ④ (정답) **f(6) = 15 + 9 + 1 = 25.**
- ⑤ 41은 f(7)의 값이다(25 + 15 + 1).

32 ② 자료구조

이 후위 표기식의 값과, 계산하는 동안 스택에 한꺼번에 쌓이는 원소의 최대 개수는?

왼쪽부터 하나씩 처리한다.

읽은 것	처리	스택	개수
9	넣기	9	1
3	넣기	9, 3	2
/	$9 \div 3 = 3$	3	1
2	넣기	3, 2	2
5	넣기	3, 2, 5	3
*	$2 \times 5 = 10$	3, 10	2
+	$3 + 10 = 13$	13	1
4	넣기	13, 4	2
-	$13 - 4 = 9$	9	1

값은 9, 최대 깊이는 3개다.

중위로 옮기면 $9 \div 3 + 2 \times 5 - 4$ 다. 후위 표기에는 괄호가 없어도 계산 순서가 하나로 정해진다.

선택지 해설

- ① 2개로는 3, 2, 5 가 한꺼번에 쌓이는 순간을 담지 못한다.
- ② (정답) **값은 9, 최대 깊이는 3개다.**
- ③ 4개까지 쌓으려면 피연산자가 네 번 이어 나와야 한다. 여기서는 세 번이 최대다.
- ④ 11은 뺄셈의 두 값을 거꾸로 놓은 값이 아니라 계산 순서를 잘못 잡은 값이다. 연산자는 **먼저 꺼낸 것이 오른쪽 피연산자**다.
- ⑤ 13은 마지막 뺄셈을 하기 직전의 값이다.

홀기마다 남은 구간을 전부 본다.

홀기	1	2	3	4	5	6	7	합
비교	7	6	5	4	3	2	1	28

$7 + 6 + \dots + 1 = 8 \times 7 \div 2 = 28$ 회다. 이 값은 **자료가 어떻게 놓여 있든 똑같다**. 이미 정렬된 배열을 넣어도 28회를 다 본다.

교환은 홀기마다 많아야 한 번, 홀기는 7번이므로 **최대 7회**다. 교환이 적은 것이 선택 정렬의 장점이다 — 비교는 $O(n^2)$ 이지만 교환은 $O(n)$ 이다.

선택지 해설

① (정답) 비교 28회 · 교환 최대 7회.

② 홀기는 8번이 아니라 7번이다. 마지막 한 원소는 저절로 제자리다.

③ 교환이 비교만큼 일어나는 것은 버블 정렬의 최악이다. 선택 정렬은 홀기당 한 번만 바꾼다.

④ 56은 8×7 이다. 같은 짝을 두 번 세지 않으므로 2로 나눈다.

⑤ 64는 8×8 이다. 자기 자신과는 비교하지 않는다.

자식 생성자는 몸통을 돌기 전에 부모 생성자를 먼저 부른다.

실행	부르는 차례	찍히는 것
new B()	B() → (감춰진 super()) → A() → B()의 몸통	A 그다음 B
new B(1)	B(1) → super(1) → A(int) → B(1)의 몸통	A1 그다음 B1

합치면 **ABA1B1** 이다.

B()에는 super()를 적지 않았지만 컴파일러가 **매개변수 없는 super()**를 자동으로 넣는다. 그래서 A(int)가 아니라 A()가 불린다. B(int)는 super(x)를 직접 적었으므로 A(int)가 불린다.

선택지 해설

① 두 객체가 서로 섞였다. new B() 가 다 끝난 뒤에 new B(1) 이 시작한다.

② new B(1) 에서 A(int)가 빠졌다. super(x)가 그것을 부른다.

③ 부모 생성자가 먼저다. 자식의 몸통은 그 뒤에 돈다.

④ 실행 순서가 뒤바뀌었다. new B() 가 먼저다.

⑤ (정답) ABA1B1.

8비트에는 값이 256가지밖에 못 담긴다. 넘치면 **256으로 나눈 나머지의 비트가 남는다**. 남은 비트를 어떻게 읽느냐가 두 자료형에서 갈린다.

	계산	남는 비트	읽는 범위	값
unsigned char	$200 + 100 = 300$	0010 1100	0 ~ 255	44
signed char	$100 + 100 = 200$	1100 1000	-128 ~ 127	-56

$300 - 256 = 44$ 다. unsigned는 0부터 세므로 그대로 44다.

200은 비트로 1100 1000 이다. signed는 맨 앞 비트를 부호로 읽으므로 $200 - 256 = -56$ 이 된다.

선택지 해설

① (정답) 44 와 -56.

- ② -100은 부호만 뒤집은 값이다. 넘친 값은 부호가 뒤집히는 것이 아니라 256을 빼는 것이다.
- ③ 200은 signed char 가 담을 수 있는 범위(-128 ~ 127) 밖이다.
- ④ 300은 unsigned char 의 범위(0 ~ 255) 밖이다. 8비트에 담기지 않는다.
- ⑤ 두 값 모두 8비트를 넘는다. 넘친 자리는 버려진다.

정 보 보 안

갑은 을이 보낸 19를 자기 비밀수 6으로 거듭제곱한다.

$19^2 = 361 = 23 \times 15 + 16 \rightarrow 16$

$19^3 = 16 \times 19 = 304 = 23 \times 13 + 5 \rightarrow 5$

$19^6 = (19^3)^2 = 5^2 = 25 = 23 + 2 \rightarrow 2$

을도 같은 값에 이른다. 갑이 보낸 8을 자기 비밀수 15로 거듭제곱하면 $8^{15} \bmod 23 = 2$ 다.

둘 다 $g^{ab} \bmod p$ 를 구하는 셈이라 순서가 달라도 값이 같다. 엇듣는 쪽은 $p \cdot g \cdot A \cdot B$ 를 다 보지만, 거기서 a나 b를 되찾으려면 **이산로그를 풀어야** 한다. 그것이 이 방식이 기대는 어려움이다.

선택지 해설

① (정답) $19^6 \bmod 23 = 2$. 갑이 보낸 8로 계산해도 $8^{15} \bmod 23 = 2$ 로 같다.

- ② 4는 $5^4 \bmod 23$ 의 값이다. 생성원을 그대로 거듭제곱했다.
- ③ 8은 갑이 **보낸 값**이다. 공유키가 아니라 공개된 값이다.
- ④ 16은 $19^2 \bmod 23$ 으로, 거듭제곱을 6이 아니라 2까지만 한 값이다.
- ⑤ 19는 을이 보낸 값이다. 그것을 자기 비밀수로 한 번 더 거듭제곱해야 한다.

복호식에 C3가 나오는 자리를 찾으면 된다. C3는 두 곳에 쓰인다.

블록	복호식	C3를 쓰나	결과
P2	$D(C2) \oplus C1$	아니다	온전
P3	$D(C3) \oplus C2$	쓴다	블록 전체가 엉킨다
P4	$D(C4) \oplus C3$	쓴다	같은 자리의 비트 하나만 뒤집힌다
P5	$D(C5) \oplus C4$	아니다	온전

D(C3)는 블록 암호를 거치므로 한 비트만 틀려도 **블록 전체가 알아볼 수 없게** 된다. 반면 P4에서 C3는 XOR로 쓰일 뿐이라 뒤집힌 그 비트 하나만 옮겨간다.

오류는 두 블록에서 멈춘다. 뒤로 무한히 번지지 않는 것이 CBC의 성질이다.

선택지 해설

- ① P4도 어긋난다. 복호할 때 C3를 XOR 값으로 쓴다.
- ② (정답) P3는 블록 전체가, P4는 비트 하나가 어긋난다.
- ③ P2는 C1과 C2만 쓴다. 오류는 **뒤로** 번지지 앞으로 가지 않는다.
- ④ P5는 C4와 C5만 쓴다. C3와 무관하다.
- ⑤ 끝까지 번지는 것은 암호화할 때의 이야기다. **복호는 두 블록에서 멈춘다.**

공개키로 잠근 것은 **짜이 되는 개인키로만** 열린다. 을의 개인키는 을만 갖고 있으므로, 을의 공개키로 감싸면 을만 풀 수 있다.

차례	갑이 하는 일	을이 하는 일
1	세션키를 하나 만든다	
2	파일을 세션키로 암호화	
3	세션키를 을의 공개키 로 암호화	
4	둘을 함께 보낸다	자기 개인키로 세션키를 푼다
5		그 세션키로 파일을 푼다

큰 파일까지 공개키로 암호화하지 않는 이유는 **느리기 때문**이다. 빠른 대칭키로 본문을 잠그고, 그 열쇠만 공개키로 감싼다 — 두 방식의 장점을 나눠 쓰는 것이다.

갑이 보냈다는 것까지 보이려면 **전자서명을 따로 붙인다**. 전자봉투는 비밀을 지키는 장치이지 신원을 밝히는 장치가 아니다.

선택지 해설

- ① 갑의 개인키로 감싼 것은 갑의 공개키로 누구나 풀 수 있다. 그것은 서명이지만 비밀 유지가 아니다.
- ② 갑의 공개키로 잠그면 갑의 개인키로만 열린다. 을이 못 푼다.
- ③ 을의 개인키는 을만 갖고 있다. 갑이 쓸 수 없다.
- ④ (정답) **을의 공개키로 감싸면 짜이 되는 을의 개인키로만** 열린다.
- ⑤ 미리 나눠 가진 대칭키가 없다. 그 키를 안전하게 나누는 문제를 풀려고 전자봉투를 쓰는 것이다.

말은 역할이 아니라 **지금 컨 역할**이 권한을 준다.

사용자	컨 역할	지금 갖는 권한	쓰기
김대리	조회자	읽기	×
이주임	편집자	읽기 · 쓰기	○
박과장	관리자	읽기 · 쓰기 · 삭제 · 권한부여	○
최사원	조회자	읽기	×

김대리는 편집자 역할을 갖고는 있지만 **켜지 않았다**. 지금 이 순간에는 쓰기를 할 수 없고, 세션에서 편집자를 켜야 할 수 있다.

이것이 최소 권한 원칙을 세션 단위로 지키는 방법이다 — 읽기만 하려고 들어왔다면 쓰기 권한은 켜 두지 않는다. 실수로든 침입자에게든 그 권한이 쓰일 일을 없앤다.

선택지 해설

- ① 이주임도 편집자를 켜 두었다. 편집자에게는 쓰기 권한이 있다.
- ② 박과장의 관리자 역할에도 쓰기가 들어 있다.
- ③ (정답) 이주임(편집자)과 박과장(관리자) 둘이다.
- ④ 김대리는 편집자 역할을 갖고 있지만 지금은 조회자만 켜다. **갖는 것과 켜는 것은 다르다.**
- ⑤ 최사원은 조회자 역할만 갖고 있다. 켜고 말고를 떠나 쓰기 권한이 없다.

세션 고정 공격은 **공격자가 미리 만들어 둔 세션 ID를 피해자에게 쥐여 주는** 수법이다. 피해자가 그 ID로 로그인하면 같은 ID를 들고 있는 공격자도 로그인된 세션을 쓰게 된다.

막는 방법은 그 반대다 — **로그인에 성공하는 순간 세션 ID를 새로 발급하고** 예전 것을 버린다. 그러면 공격자가 쥐여 준 ID는 로그인과 이어지지 않는다.

「그대로 이어 쓴다」는 것은 대책이 아니라 **취약점 그 자체**다.

공격	핵심 대책
SQL 인젝션	매개변수화 질의 — 값이 코드로 읽히지 않게
XSS	출력 이스케이프 — 값이 태그로 읽히지 않게
CSRF	토큰 — 남의 사이트에서 만든 요청을 가려내게
경로 조작	정규화 후 허용 범위 확인
세션 고정	로그인 성공 시 세션 ID 재발급

선택지 해설

- ① 맞다. 값을 이어 붙이지 않으면 입력이 질의문의 구조를 바꿀 수 없다.
- ② 맞다. 저장할 때가 아니라 ****내보낼 때**** 이스케이프하는 것이 맞다.
- ③ 맞다. 남의 사이트에서는 그 토큰을 알아낼 수 없다.
- ④ 맞다. `./` 를 지우는 것만으로는 모자라고 정규화한 뒤 확인해야 한다.
- ⑤ (정답) 틀렸다. 그대로 이어 쓰면 오히려 세션 고정을 당한다. 로그인에 성공하면 세션 ID를 ****새로 발급****해야 한다.